Labo 12 — Questions

Question 1 [Compréhension]

Un Deployment `api` à trois réplicas a été créé, puis mis à jour deux fois. Combien de ReplicaSets existent, combien de Pods, et que montre `kubectl get rs` ? Pourquoi les anciens ReplicaSets ne sont-ils pas supprimés, et quel réglage borne leur nombre ?

Question 2 [Diagnostic]

```
spec:
  replicas: 2
  selector:
    matchLabels:
      app: api
  template:
    metadata:
      labels:
        app: backend
```

`kubectl apply` refuse ce Deployment. Donnez le message d'erreur en substance, expliquez la règle violée, et dites ce qui se passerait si le cluster l'acceptait.

Question 3 [Prédiction]

Un Deployment à trois réplicas, stratégie par défaut, passe de `api:1.0` à `api:1.1`. Décrivez la séquence des Events `ScalingReplicaSet` (combien de Pods de chaque version à chaque étape), et expliquez pourquoi le cluster ne crée pas directement trois nouveaux Pods.

Question 4 [Diagnostic]

```
$ kubectl set image deployment/api api=localhost/api:bad
$ kubectl get pods
NAME                   READY   STATUS    RESTARTS      AGE
api-6c7bb54fc5-c2lhj   1/1     Running   0             32s
api-6c7bb54fc5-km2h6   1/1     Running   0             40s
api-6c7bb54fc5-s9c8f   1/1     Running   0             40s
api-6c87f96dd5-4v6s6   0/1     Error     2 (24s ago)   25s
```

Pourquoi n'y a-t-il qu'un seul Pod de la nouvelle version, pourquoi les trois anciens tournent-ils toujours, et que répond `kubectl rollout status` ? Donnez les deux commandes qui révèlent la cause puis réparent.

Question 5 [Analyse]

Un collègue règle une `livenessProbe` qui interroge `/actuator/health`, lequel renvoie `DOWN` dès que la base de données est injoignable. La base tombe cinq minutes. Décrivez ce qui arrive à l'API, pourquoi c'est pire que sans sonde, et ce qu'il fallait faire.

Question 6 [Prédiction]

Dans un Deployment à trois réplicas sains, vous modifiez le template pour que la `readinessProbe` vise le port `8081` (où rien n'écoute). Que montrent `kubectl get pods` et `kubectl get deployment api` trente secondes plus tard ? Le service répond-il encore ? Même question si c'est la `livenessProbe` que vous aviez cassée.

Question 7 [Compréhension]

Expliquez ce qu'est `CrashLoopBackOff` : est-ce un état de l'application, du conteneur ou du kubelet ? Pourquoi les relances s'espacent-elles ? Quelle commande montre la cause, et pourquoi `kubectl logs` sans option peut-il n'afficher rien d'utile ?

Question 8 [Analyse]

Comparez le `HEALTHCHECK` du labo 10 et une `readinessProbe httpGet` : qui exécute la vérification, où doit se trouver l'outil (`wget`, `curl`), quel effet a un échec, et lequel des deux fonctionne avec une image distroless ?

Question 9 [Diagnostic]

`kubectl rollout history deployment/api` affiche les révisions `2, 4, 6, 7` : les numéros `1, 3, 5` manquent. Expliquez d'où viennent ces trous (indice : chaque `rollout undo` crée une révision) et pourquoi ce n'est pas un signe de problème.

Question 10 [Analyse]

Un Deployment a `maxSurge: 0` et `maxUnavailable: 1` ; un autre a `maxSurge: 1` et `maxUnavailable: 0`. Pour trois réplicas, décrivez la mise à jour dans chaque cas (nombre de Pods au pic et au creux), et dites lequel convient à un cluster où les nœuds sont pleins et lequel à un service qui ne doit jamais perdre de capacité.

Question 11 [Diagnostic]

Après `kubectl scale deployment api --replicas=5`, deux Pods restent `Pending` indéfiniment, les trois autres sont `Running`. Quelle est la cause la plus probable sur un k3s mono-nœud, où la lire, et que se passera-t-il si vous appliquez un `rollout` dans cet état ?

Question 12 [Analyse]

Un développeur contourne Kubernetes : il fait `kubectl exec` dans un Pod, modifie un fichier de configuration et redémarre le processus à la main. Décrivez ce qui arrive au prochain rollout, au prochain redémarrage du Pod, et sur les deux autres réplicas. Qu'aurait-il dû faire (deux objets des labos suivants) ?

Question 13 [Analyse]

Un `Pod` seul écrit dans un manifeste (`kind: Pod`) est appliqué en production « parce qu'on n'a qu'une instance ». Donnez quatre choses que ce choix empêche, par rapport à un Deployment à `replicas: 1`, et le seul cas où un Pod nu est le bon objet.